

Mario Bros Gesture Controller

Controla Super Mario Bros con gestos de mano
usando tu cámara web · OpenCV · MediaPipe · PyAutoGUI

Python 3.8+

OpenCV

MediaPipe

FCEUX NES

Versión 1.0 · 2026 · github:<https://github.com/SV001Noire>

■ Tabla de Contenido

1.	Descripción del proyecto
2.	Requisitos del sistema
3.	Instalación paso a paso
4.	Estructura del repositorio
5.	Configuración de FCEUX
6.	Gestos y controles
7.	Parámetros configurables
8.	Resolución de problemas
9.	Preguntas frecuentes
10.	Créditos y licencia

■ 1. Descripción del Proyecto

Mario Bros Gesture Controller es un proyecto de visión por computadora que te permite jugar Super Mario Bros en el emulador FCEUX usando únicamente gestos de tu mano frente a la cámara web. Sin joystick, sin teclado — solo tus manos.

El sistema usa **MediaPipe Hands** para detectar y rastrear los 21 landmarks de cada mano en tiempo real, calcula distancias entre dedos para reconocer gestos, y los traduce en pulsaciones de teclas mediante **PyAutoGUI**.

¿Cómo funciona internamente?

- La cámara captura ~30 fps del video en vivo
- MediaPipe detecta hasta 2 manos simultáneas con modelo de baja complejidad
- Se calculan distancias normalizadas entre landmarks (pulgar, índice, muñeca)
- Un árbol de decisiones traduce los gestos a acciones de teclado
- El salto usa keyDown/keyUp con tiempo dinámico según duración del gesto

■ ■ 2. Requisitos del Sistema

Hardware

- Cámara web (integrada o USB) — resolución mínima 640×480
- Procesador: Intel Core i5 / AMD Ryzen 5 o superior
- RAM: 4 GB mínimo (8 GB recomendado)

Software

- Python 3.8 o superior
- FCEUX 2.6+ con Super Mario Bros (ROM .nes)
- Sistema operativo: Windows 10/11 (recomendado), Linux, macOS

Librerías de Python

Librería	Versión mínima	Comando de instalación
opencv-python	4.x	<code>pip install opencv-python</code>
mediapipe	0.10+	<code>pip install mediapipe</code>
pyautogui	0.9+	<code>pip install pyautogui</code>
numpy	1.21+	<code>pip install numpy</code>
mss	9.0+	<code>pip install mss</code>
pywin32	305+	<code>pip install pywin32</code> (solo Windows)

■ Puedes instalar todas las librerías de una vez con el archivo `requirements.txt` incluido en el repositorio.

■ 3. Instalación Paso a Paso

Paso 1 — Clonar el repositorio

```
git clone https://github.com/tu-usuario/mario-gesture-controller.git cd
mario-gesture-controller
```

Paso 2 — Crear entorno virtual (recomendado)

```
python -m venv venv # Windows: venv\Scripts\activate # Linux / macOS: source
venv/bin/activate
```

Paso 3 — Instalar dependencias

```
pip install -r requirements.txt
```

Paso 4 — Verificar instalación

```
python -c "import cv2, mediapipe, pyautogui, mss; print('OK')"
```

Si ves **OK** en la consola, todo está instalado correctamente.

Paso 5 — Ejecutar

```
python mario_gestures.py
```

■ **Asegúrate de que FCEUX esté abierto y con el juego cargado antes de ejecutar el script. El programa espera 2 segundos al inicio para darte tiempo de posicionarte.**

■ 4. Estructura del Repositorio

```
mario-gesture-controller/ ■■■ mario_gestures.py # Controlador principal (gestos →
teclado) ■■■ requirements.txt # Dependencias del proyecto ■■■ README.pdf # Este
documento ■■■ README.md # Versión Markdown para GitHub
```

■ ■ 5. Configuración de FCEUX

Mapeo de teclas en FCEUX

Ve a **Config** → **Input** → **Gamepad 1** y asigna las teclas así:

Botón NES	Tecla en FCEUX
Right (→)	D
Left (←)	A
Up (↑)	W
Down (↓)	S
A (salto)	Spacebar
B (correr)	Shift
Start	Enter

Select	Backspace
--------	-----------

■ 6. Gestos y Controles

El sistema reconoce los siguientes gestos en tiempo real. Los gestos se detectan midiendo distancias entre landmarks de MediaPipe:

Gesto	Mano	Acción	Tecla
Mano derecha visible	Derecha	Moverse a la derecha	D
Mano izquierda visible	Izquierda	Moverse a la izquierda	A
Pellizco (juntar dedos)	Cualquiera	Cargar salto	—
Soltar pellizco	Cualquiera	Ejecutar salto	Space
Mano abierta	Cualquiera	Correr	Shift
Puño cerrado	Cualquiera	Agacharse	S
Ambas manos	Cualquiera	Activar/Desactivar “Run”	—

Detalle del salto dinámico

El salto usa un sistema de **carga por tiempo**: cuanto más tiempo mantienes los dedos juntos (pellizco), más alto salta Mario al soltarlos. Una barra visual en la ventana de preview muestra la carga acumulada.

- **Toque rápido** (menos de 0.1 s) → salto mínimo
- **Hold medio** (0.2–0.3 s) → salto estándar
- **Hold completo** (0.5 s+) → salto máximo

■ 7. Parámetros Configurables

Todos los parámetros se encuentran al inicio del archivo `mario_gestures.py` y pueden modificarse sin tocar la lógica del programa:

Umbral de gestos

```
JUMP_THRESHOLD = 0.05 # distancia máx. para 'dedos juntos' (salto) RUN_THRESHOLD = 0.15
# distancia mín. para 'mano abierta' (correr) CROUCH_THRESHOLD = 0.03 # distancia mín.
para 'puño cerrado' (agacharse)
```

Configuración del salto

JUMP_MIN_HOLD	JUMP_MAX_HOLD	Resultado
0.05 s	0.3 s	Saltos cortos y rápidos
0.05 s	0.5 s	Saltos completos (recomendado)
0.1 s	0.6 s	Saltos muy altos / lentos

Cooldown entre saltos

```
JUMP_COOLDOWN = 0.5 # segundos mínimos entre saltos consecutivos
```

■ Si el juego detecta saltos involuntarios, aumenta `JUMP_THRESHOLD` a 0.07 o más. Si no detecta los saltos bien, bájalo a 0.03.

■ 8. Resolución de Problemas

Mario no se mueve / el juego no responde

- Asegúrate de que FCEUX esté en primer plano al iniciar el script
- Instala pywin32 (pip install pywin32) para foco automático de ventana
- Desactiva 'PAUSE_ON_UNFOCUS' en Config → Emulation de FCEUX

Detección de manos poco precisa

- Mejora la iluminación — evita contraluz directo a la cámara
- Usa un fondo uniforme (pared blanca o lisa funciona mejor)
- Ajusta min_detection_confidence en el código (0.5–0.7)

Alto uso de CPU / lag

- El modelo model_complexity=0 ya es el más ligero disponible
- Reduce la resolución de captura a 320×240 si el CPU es lento
- Cierra otras aplicaciones pesadas mientras juegas

Error: No module named 'X'

```
pip install opencv-python mediapipe pyautogui mss pywin32
```

■ 9. Preguntas Frecuentes

P: ¿Funciona con cualquier juego de NES?

R: Sí, siempre que el juego use las mismas teclas. Puedes cambiar las constantes KEY_RIGHT, KEY_LEFT, KEY_JUMP, etc. al inicio del archivo para cualquier mapeo.

P: ¿Necesito una cámara especial?

R: No. Cualquier webcam USB o integrada de portátil funciona. Resolución mínima recomendada: 640×480 a 30 fps.

P: ¿Funciona en Linux o macOS?

R: Sí, pero pywin32 solo está disponible en Windows. En Linux/macOS el foco automático de ventana no funcionará — deberás hacer clic en FCEUX manualmente antes de iniciar.

P: ¿Puedo usar ambas manos al mismo tiempo?

R: Sí. El sistema detecta hasta 2 manos. Mano derecha = moverse a la derecha, mano izquierda = moverse a la izquierda. Los gestos de salto/correr/agacharse funcionan con cualquiera de las dos.

P: ¿Cómo calibro los colores si la detección falla?

R: Agrega print(frame_hsv[y, x]) en el loop para leer el valor HSV exacto de cualquier píxel y ajusta los rangos correspondientes.

■ 10. Créditos y Licencia

Este proyecto fue construido con las siguientes tecnologías open source:

Tecnología	Uso en el proyecto	Licencia
MediaPipe	Detección de manos y landmarks	Apache 2.0
OpenCV	Captura y procesamiento de video	Apache 2.0
PyAutoGUI	Simulación de teclado	BSD
mss	Captura de pantalla (AI agent)	MIT
NumPy	Operaciones matriciales	BSD
FCEUX	Emulador NES	GPL v2

Este proyecto se distribuye bajo licencia **MIT**. Puedes usarlo, modificarlo y distribuirlo libremente citando al autor original.